Learning-Native Adaptive Software Framework (LNASF)

Research Positioning and Candidate Component Contract

Peyman Salimi
salimipeyman@gmail.com
Alternate: pmnslm6344@gmail.com

Research draft v0.2
October 11, 2026

Contents

Keywords **3**

Abstract **3**

1 Motivation **3**

2 Core Idea **4**

3 What LNASF Is and Is Not **4**

4 Learning as a Software-Component Capability **4**

5 Native Learning **4**

6 Scratch Learning **5**

7 Prediction Is Not Action **5**

8 Confidence and Uncertainty **5**

9 Deterministic Fallback **6**

10 Levels of Adaptation **6**
10.1 Parameter Adaptation 6
10.2 Strategy Adaptation 6
10.3 Structural Adaptation 6

11 Learning Modes **6**
11.1 Passive 6
11.2 Advisory 6
11.3 Adaptive 6
11.4 Autonomous 6

12 Learning Scope **6**
12.1 Local Learning 6
12.2 Shared Learning 7
12.3 Hybrid Learning 7

13 Feedback Loop **7**

14 Model Drift and Learning Failure **7**

15 Privacy **7**

16 Frontend Applications **7**

17 Backend Applications **8**

18 Runtime and Infrastructure Applications **8**

19 Independence from Programming Language **8**

20 Independence from Learning Algorithm **8**

21 Developer Model **8**

22 Evaluation **8**

23 Prior Art and Related Work **9**

24 Position Relative to Prior Art **9**
24.1 Candidate Contribution: LNASF Component Contract 9

25 Research Hypothesis **10**

26 Potential Software Family **10**

27 Development Roadmap **10**

28 Limitations and Open Questions **11**

29 Design Principles **11**

30 Scope of This Disclosure **11**

31 Citation **11**

32 References **11**

Keywords

Machine Learning; Artificial Intelligence; Self-Adaptive Software; Adaptive Software; Online Learning; Runtime Adaptation; Learning-Enabled Software; Learning-Native Software Components; Adaptive Software Components; Software Libraries; Software Frameworks; Runtime Intelligence; Native Machine Learning; Scratch Machine Learning; Predictive Software; Adaptive Systems; Feedback-Driven Software.

Abstract

Software has traditionally been built around rules that are designed in advance. Developers decide how a library should cache data, how a framework should schedule work, how a runtime should handle failures, or how a user interface should load resources.

That approach is predictable and often effective. However, real-world software rarely operates under stable conditions. Users behave differently over time, workloads change, network conditions fluctuate, and the cost of different execution strategies can vary significantly.

This document proposes the **Learning-Native Adaptive Software Framework (LNASF)**, a general architectural framework for building software components that can learn from their own runtime experience and use that knowledge to adapt future behavior.

The framework is intended for reusable software components, including libraries, frameworks, runtimes, developer tools, frontend systems, backend systems, and infrastructure components.

The central idea is a continuous feedback loop:

$$Observe \rightarrow Learn \rightarrow Predict \rightarrow Decide \rightarrow Adapt \rightarrow Measure \rightarrow Learn$$

LNASF does not claim to introduce machine learning, self-adaptive software, runtime adaptation, online learning, or learning-enabled components as new concepts. These areas have extensive prior research and established techniques.

The candidate contribution investigated in this research draft is an explicit component-boundary contract for learning-enabled software. It specifies observation, model updates, prediction evidence, adaptation admission, permitted actions, deterministic fallback, and outcome evaluation. It does not claim to originate any individual building block; whether this contract is distinct from prior component-based and self-adaptive frameworks remains to be established.

1. Motivation

Most software libraries and frameworks behave according to rules that are determined before deployment.

Examples include:

```
timeout = 500 ms
retryCount = 3
cacheTTL = 5 min
prefetch = enabled
concurrency = 4
```

These values and strategies may work well in development and testing, but they are not necessarily optimal under every runtime condition.

Users change their behavior. Workloads change. Network conditions fluctuate. Failure patterns evolve. Resource availability varies. A policy that was reasonable yesterday may not be the best policy tomorrow.

LNASF asks a practical engineering question:

> Can a reusable software component observe its runtime experience, learn from that experience, and use the learned knowledge to make better decisions later?

2. Core Idea

The central concept of LNASF is a **Learning-Native Adaptive Software Component (LNASC)**.

Such a component has a normal deterministic implementation, but also contains a learning capability that can use runtime observations to improve selected decisions.

A simplified representation is:

$$Component = Deterministic\ Baseline + Learning\ Layer + Adaptive\ Policy$$

The learning layer does not replace the original software logic. It provides an additional source of information that can influence selected runtime decisions.

The fundamental learning loop is:

$$O_t \rightarrow M_t \rightarrow \hat{Y}_{t+1} \rightarrow D_t \rightarrow A_t \rightarrow R_t \rightarrow M_{t+1}$$

The model may be updated using:

$$M_{t+1} = U(M_t, O_t, R_t)$$

3. What LNASF Is and Is Not

LNASF is an architectural proposal rather than a single machine-learning algorithm or product.

It is intended to provide a common way of reasoning about what a software component should observe, learn, predict, decide, constrain, and evaluate.

LNASF does not claim to invent machine learning, online learning, self-adaptive software, autonomic computing, MAPE-K, runtime adaptation, predictive prefetching, adaptive caching, adaptive scheduling, or learning-enabled components. These areas are treated as prior art and foundations.

4. Learning as a Software-Component Capability

LNASF treats learning as an explicit capability of a reusable software component.

```
Application
   -> Reusable Software Component
      -> Deterministic Behavior
      -> Learning Capability
         -> Runtime Experience
            -> Adaptation
```

The intention is not to turn every component into an autonomous system. The intention is to make adaptive capability a deliberate architectural choice.

5. Native Learning

One of the design principles of LNASF is **native learning**.

The learning layer should, as far as practical, operate within the language and runtime ecosystem of the host software.

```
TypeScript Library -> TypeScript Learning Engine
C# Library -> C# Learning Engine
Go Library -> Go Learning Engine
Rust Library -> Rust Learning Engine
Python Library -> Python Learning Engine
```

The baseline architecture should not require a different programming language, an external inference service, or a remote machine-learning runtime merely to provide adaptive behavior.

```
Host Software
    -> Native Learning Layer
       -> Prediction
          -> Decision
             -> Adaptation
```

Native learning is a design principle proposed by LNASF. It is not presented as a historical first.

6. Scratch Learning

A reference implementation should not require a specialized machine-learning framework to execute the essential learning mechanism.

Learning methods may be constructed from arithmetic, probability, statistics, linear algebra, numerical methods, optimization, and data structures.

For example:

$$P(j|i) = \frac{N(i,j)}{N(i)}$$

or an online parameter update such as:

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

The purpose is transparency and portability of the learning mechanism rather than avoidance of every standard-library function.

7. Prediction Is Not Action

A central LNASF principle is:

$$Prediction \neq Action$$

The decision process may be modeled as:

$$Decision = f(Prediction, Confidence, ExpectedBenefit, ExpectedCost, Constraints)$$

One possible utility formulation is:

$$U(a|x) = E[Benefit(a)|x] - E[Cost(a)|x]$$

with:

$$a^* = \arg\max_a E[U(a|x)]$$

subject to applicable constraints.

8. Confidence and Uncertainty

For example:

$$P(A|X) = 0.54$$

may not justify an adaptation, while:

$$P(A|X) = 0.96$$

may justify it, depending on the cost and risk.

An adaptive component may therefore use:

$$Confidence \geq \tau$$

before enabling a given adaptation.

9. Deterministic Fallback

A learning-enabled component must remain useful when learning is unavailable or unreliable.

```
Learned Policy
    -> Validate
       -> Safe? ---- Yes -> Adaptive Action
            \---- No -> Deterministic Baseline
```

Conceptually:

$$Behavior = Deterministic\ Baseline + Optional\ Adaptive\ Intelligence$$

10. Levels of Adaptation

10.1 Parameter Adaptation

The component preserves its algorithm or strategy but changes parameters such as timeout, retry delay, cache TTL, debounce interval, concurrency limit, or prefetch threshold.

10.2 Strategy Adaptation

The component selects between predefined strategies, for example aggressive, balanced, and conservative prefetching.

10.3 Structural Adaptation

The component changes an execution path, composition, or selected implementation. This is treated as an advanced capability.

11. Learning Modes

11.1 Passive

Observe and learn without changing behavior.

11.2 Advisory

Generate predictions or recommendations while leaving the final decision to the application or developer.

11.3 Adaptive

Automatically apply adaptations permitted by policy.

11.4 Autonomous

Continuously optimize behavior within explicit safety, resource, and policy constraints.

12. Learning Scope

12.1 Local Learning

The model belongs to a component, application, session, process, device, or user.

12.2 Shared Learning

Observations from multiple instances contribute to a shared model.

12.3 Hybrid Learning

A shared model provides a baseline while local models adapt to local conditions:

$$M = M_{global} + M_{local}$$

13. Feedback Loop

The observed result of an action becomes evidence for subsequent learning:

```
Observation
   -> Learning
      -> Prediction
         -> Decision
            -> Action
               -> Outcome
                  -> Evaluation
                     -> Model Update
                        -> Observation
```

14. Model Drift and Learning Failure

A learning-enabled component should consider cold start, insufficient data, concept drift, distribution changes, prediction error, adaptation instability, model degradation, and unexpected outcomes.

15. Privacy

For client-side systems, local-first learning is an important option:

```
User Behavior
   -> Local Observation
      -> Local Model
         -> Local Adaptation
```

Remote aggregation should be optional, explicit, and governed by privacy requirements.

16. Frontend Applications

Possible applications include predictive prefetching, adaptive loading, adaptive caching, request prioritization, adaptive debouncing, learnable state behavior, and resource scheduling.

For example:

$$P(Page_B|Page_A) = 0.87$$

A client can combine that estimate with confidence and network cost before deciding whether to prefetch Page B. Predictive prefetching itself is established prior art and is included here only as an example application.

17. Backend Applications

A backend component may estimate:

$$P(success|errorType, service)$$

and choose among retry, delayed retry, or fail-fast behavior. Other applications include adaptive timeout, adaptive retry, adaptive concurrency, workload-aware scheduling, cache optimization, and resource allocation.

18. Runtime and Infrastructure Applications

Runtime and infrastructure components may observe CPU, memory, I/O latency, request rate, queue depth, failure rate, and workload patterns. A general workload loop can be expressed as:

$$Workload_t \rightarrow Prediction_t \rightarrow SchedulingDecision_t$$

19. Independence from Programming Language

The architecture is intentionally language-independent and can be implemented in TypeScript, C#, Java, Go, Rust, Python, and other ecosystems. The architecture is shared while implementation details remain native to the host environment.

20. Independence from Learning Algorithm

Potential implementations include frequency models, conditional probability, moving averages, Bayesian updating, Markov models, online gradient methods, contextual bandits, and other suitable methods.

21. Developer Model

An LNASF-enabled component should make explicit what it observes, learns, predicts, and changes, what constraints limit adaptation, what happens under low confidence, what the deterministic fallback is, and how success is measured.

22. Evaluation

An adaptive component should be compared against a deterministic baseline. Potential metrics include:

- Latency
- CPU utilization
- Memory usage
- Network cost
- Error rate
- Prediction accuracy
- Adaptation overhead
- False-adaptation rate

A general formulation is:

$$NetUtility = Benefit - Cost - AdaptationOverhead$$

23. Prior Art and Related Work

LNASF is situated within established research on autonomic computing and self-adaptive systems [1,2], machine learning in feedback-loop-based adaptation [3], online runtime learning and component assembly [4], ML-enabled self-organizing components [5], component-based ML abstractions [6,7], adaptive feedback-loop architectures [8], reusable architecture-based self-adaptation infrastructure [11], and the FORMS unifying reference model for distributed self-adaptive systems [10].

REX directly combines online learning with runtime adaptation of software components. ML-DEECo and subsequent work integrate machine learning into component-based adaptive systems, while HAFLoop studies reusable architectures for adaptation of feedback loops. These works are close prior art. LNASF therefore does not claim to originate online learning, runtime adaptation, component-level learning, generic feedback loops, or safety constraints as isolated concepts.

This is a focused preliminary comparison, not an exhaustive systematic literature review. The full-text comparisons and candidate evaluation plan are documented in `RESEARCH-POSITIONING.md`.

24. Position Relative to Prior Art

LNASF does not claim to invent machine learning, online learning, self-adaptive software, autonomic computing, MAPE-K, runtime learning, adaptive feedback loops, predictive prefetching, adaptive caching, model uncertainty, safety constraints, or deterministic fallback as isolated ideas.

The candidate contribution is a proposed reusable and testable component-boundary contract that states what a component observes and learns, how predictions and their evidence are represented, how a separate gate admits or rejects candidate adaptations, which actions are allowed, what baseline behavior is retained, and how outcomes and adaptation overhead are measured.

24.1 Candidate Contribution: LNASF Component Contract

For a learning-enabled component C, define the proposed contract:

$$\mathrm{LCC}(C) = (B, O, M, P, G, A, F, E)$$

where B is deterministic baseline behavior; O the observation interface and collection boundaries; M the model and update procedure; P the prediction interface including uncertainty or evidence sufficiency; G an adaptation-admission gate separate from the predictor; A the permitted actions and resource bounds; F the fallback or abstention behavior; and E outcome evaluation and feedback.

A candidate gate is:

$$G_t(a) = [q_t \geq \epsilon] \wedge [S(a, o_t) = \text{true}] \wedge [\widehat{\Delta U}_t(a) - C_{\text{adapt},t} > \lambda].$$

The component applies the adaptive action only if the gate passes; otherwise it uses $B(o_t)$ or returns an advisory result. The thresholds are application-specific. A confidence score must not be described as a calibrated probability without calibration evidence.

This proposal treats the explicit contract and its conformance/evaluation obligations as the candidate research contribution—not the invention of the individual components. Whether the contract is sufficiently different from REX, ML-DEECo, ML abstraction frameworks, and reusable adaptive feedback loops remains an open question. FORMS [10] and Rainbow [11] mean that LNASF cannot claim distinction merely because it presents a reusable or unifying adaptation architecture.

25. Research Hypothesis

> An explicit LNASF Component Contract—with inspectable observation, model-update, prediction-evidence, adaptation-admission, allowed-action, fallback, and evaluation obligations—can make adaptive behavior easier to inspect and test than implementations where these concerns remain implicit, while preserving safe fallback and accounting for adaptation overhead.

This is an empirical hypothesis, not an established result. It should be evaluated by contract-conformance checks, controlled comparisons with a deterministic baseline, component-level ablations, and measurements of both outcome quality and adaptation cost.

26. Potential Software Family

```
Learning-Native Software
|
+-- Frontend
| +-- Predictive Prefetch
| +-- Adaptive Loading
| +-- Adaptive Cache
| +-- Learnable State
|
+-- Backend
| +-- Adaptive Retry
| +-- Adaptive Timeout
| +-- Adaptive Scheduling
| +-- Adaptive Concurrency
|
+-- Runtime
| +-- Workload Prediction
| +-- Resource Allocation
| +-- Execution Adaptation
|
+-- Infrastructure
    +-- Failure Prediction
    +-- Recovery Policies
    +-- Resource Optimization
```

27. Development Roadmap

1. Concept: formalize terminology, boundaries, principles, and architecture.
2. Mathematical Learning Core: implement simple learning mechanisms from fundamental primitives.
3. Reference Implementation: build the first learning-native component in TypeScript, with React as an initial demonstration.
4. Evaluation: compare adaptive and deterministic behavior under realistic workloads.
5. Advanced Learning: explore Bayesian models, Markov models, online learning, contextual models, and bandit-based methods.
6. Additional Ecosystems: investigate C#, Go, Rust, Python, and Java implementations.
7. Framework and Runtime Applications: investigate larger libraries, frameworks, runtimes, and infrastructure systems.

28. Limitations and Open Questions

Open questions include cold start, model drift, adaptation stability, learning cost, reproducibility, safety guarantees, privacy, and cross-language consistency.

29. Design Principles

1. Learning is a capability.
2. Native execution is preferred.
3. Reference learning cores should be understandable from fundamental primitives.
4. Prediction is not action.
5. Adaptation is constrained.
6. Deterministic fallback is mandatory.
7. Feedback matters.
8. Learning should be measurable.

30. Scope of This Disclosure

This file is an unpublished Version 0.2 research working draft dated October 11, 2026. It adds a candidate component contract and preliminary prior-art positioning. It is not a compiled or reviewed publication PDF; its novelty claim remains unvalidated.

This document is a technical disclosure. It is not legal advice and does not by itself create exclusive rights in abstract ideas or guarantee patentability.

31. Citation

Salimi, Peyman. (2026). *Learning-Native Adaptive Software Framework (LNASF): Research Positioning and Candidate Component Contract*, research working draft v0.2.

32. References

1. Kephart, J. O., & Chess, D. M. (2003). The Vision of Autonomic Computing. *IEEE Computer*, 36(1), 41–50. DOI: 10.1109/MC.2003.1160055.
2. Wong, T., Wagner, M., & Treude, C. (2022). Self-adaptive systems: A systematic literature review across categories and domains. *Information and Software Technology*, 148, Article 106934. DOI: 10.1016/j.infsof.2022.106934.
3. Gheibi, O., Weyns, D., & Quin, F. (2022). Applying Machine Learning in Self-Adaptive Systems: A Systematic Literature Review. *ACM Transactions on Autonomous and Adaptive Systems*. DOI: 10.1145/3469440. Preprint: arXiv:2103.04112 (2021).
4. Porter, B., Grieves, M., Rodrigues Filho, R., & Leslie, D. (2016). REX: A Development Platform and Online Learning Approach for Runtime Emergent Software Systems. *Proceedings of OSDI 2016*, 333–348.
5. Töpfer, M., Abdullah, M., Kruliš, M., Bureš, T., & Hnětynka, P. (2022). ML-DEECo: A Machine-Learning-Enabled Framework for Self-organizing Components. *ACSOS 2022 Companion*. DOI: 10.1109/ACSOSC56246.2022.00033.
6. Töpfer, M., Abdullah, M., Bureš, T., Hnětynka, P., & Kruliš, M. (2023). Machine-learning abstractions for component-based self-optimizing systems. *International Journal on Software Tools for Technology Transfer*, 25, 717–731. DOI: 10.1007/s10009-023-00726-x.
7. Abdullah, M., Töpfer, M., Bureš, T., Hnětynka, P., Kruliš, M., & Pläšil, F. (2023). Introducing Estimators—Abstraction for Easy ML Employment in Self-adaptive Architectures. *ECSA 2022 Tracks and Workshops*, 370–385. DOI: 10.1007/978-3-031-36889-9_25.

8. Zavala, E., Franch, X., Marco, J., & Berger, C. (2020). HAFLoop: An architecture for supporting Highly Adaptive Feedback Loops in self-adaptive systems. *Future Generation Computer Systems*, 105, 607–630. DOI: 10.1016/j.future.2019.12.026.
9. Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, 24(3), 45–77. DOI: 10.2753/MIS0742-1222240302.
10. Weyns, D., Malek, S., & Andersson, J. (2012). FORMS: Unifying Reference Model for Formal Specification of Distributed Self-adaptive Systems. *ACM Transactions on Autonomous and Adaptive Systems*, 7(1), Article 8, 1–61. DOI: 10.1145/2168260.2168268.
11. Cheng, S.-W., Huang, A.-C., Garlan, D., Schmerl, B., & Steenkiste, P. (2004). Rainbow: Architecture-Based Self-Adaptation with Reusable Infrastructure. *IEEE Computer*, 37(10), 46–54. DOI: 10.1109/MC.2004.175.